

Energy-Efficient Computation Peer Offloading in Satellite Edge Computing Networks

Xinyuan Zhang , Jiang Liu , Ran Zhang , *Member, IEEE*, Yudong Huang , Jincheng Tong , Ning Xin ,
Liang Liu , and Zehui Xiong , *Member, IEEE*

Abstract—Recently, MEC has been integrated with satellite networks to process remote terrestrial computation tasks with superior coverage and delay. Since single satellite computation is hard to tackle spatially uneven computation workloads, computation peer offloading among multiple satellites is urgently needed to further improve service quality and resource utilization. However, considering limited resources, deficient energy, and costly overheads of communication and computation, how to enable efficient offloading cooperation in the time-varying satellite networks is a significant challenge. In this paper, we first design a satellite peer offloading scheme, where offloading is performed along multi-hop paths to explore collaborative computing capabilities. Second, we formulate the Multi-Hop Satellite Peer offloading (MHSPO) problem, aiming to jointly minimize the delay and energy consumption under system resources and backlog constraints. Then, to adapt to the network dynamics, the decision-making process with uncertain future workloads is optimized by leveraging the delayed online learning method under the Lyapunov framework. Finally, we develop a practical online distributed algorithm to solve the MHSPO problem, which is proven to achieve close-to-optimal performance. Extensive simulations show that multi-hop peer offloading among satellites improves edge computing performance efficiently.

Index Terms—Satellite networks, edge computing, load balancing, energy efficiency, peer-to-peer offloading.

I. INTRODUCTION

IN PAST decades, satellite networks have long been considered a seamless global and disaster-resistant communication

Manuscript received 2 September 2022; revised 3 February 2023; accepted 18 April 2023. Date of publication 25 April 2023; date of current version 6 March 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62171064, in part by China Postdoctoral Science Foundation under Grant 2021M700515, and in part by the State Scholarship Fund of China Scholarship Council under Grant 202106470020. Recommended for acceptance by M. Atiquzzaman. (*Corresponding author: Jiang Liu.*)

Xinyuan Zhang and Yudong Huang are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: zhangxinyuan0181@bupt.edu.cn; hyduni@bupt.edu.cn).

Jiang Liu and Ran Zhang are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with Purple Mountain Laboratories, Nanjing 211111, China (e-mail: liujiang@bupt.edu.cn; zhangran@bupt.edu.cn).

Jincheng Tong, Ning Xin, and Liang Liu are with the China Academy of Space Technology, Beijing 100876, China (e-mail: 30755497@qq.com; xinling7@sina.com; liuliang1945@buaa.edu.cn).

Zehui Xiong is with the Information Systems Technology and Design (ISTD) Pillar, Singapore University of Technology and Design, Singapore 487372 (e-mail: zehui_xiong@sutd.edu.sg).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TMC.2023.3269801>, provided by the authors.

Digital Object Identifier 10.1109/TMC.2023.3269801

paradigm [1], [2], [3], [4]. Recently, the enhanced on-board processing capabilities are shifting the role of satellites from transport relay to in-orbit computing, prompting the integration of multi-access edge computing (MEC) and satellite networks [5], [6], [7], [8]. The in-orbit computing allows directly handling tasks on satellites, preventing the long propagation delay from satellites to the ground data centers and the congestion of the downlinks with limited bandwidth [9], [10], [11]. This opens up new possibilities for fast decision-making in Internet of Remote Things (IoRT) services [12] and eases the burden of data return in space-native data processing services [11].

In response to the above trends and benefits, many works have been done towards in-orbit computing, such as architecture design [5], [6], [9], [10], [11], network element deployments [8], and computation offloading schemes [7], [13], [14], [15], [16], [17], [18]. However, most of them only consider single satellite computation. Specifically, users offload tasks to satellites within line of sight. Each satellite individually processes all its received requests rather than offloading them to neighbors, resulting in high workloads in densely populated areas and resource waste in sparsely populated areas. Therefore, multiple satellite computation is essential to improve service quality and resource utilization.

Although some works have studied cooperative offloading in terrestrial MEC networks, such as data centers [19], [20], edge clouds [21], [22], [23], [24], [25], mobile edge clusters [26], [27], [28], [29], and cellular networks [30], [31], they do not apply to satellite networks. Satellite networks have distinct channel characteristics and face more complex mobility issues compared to terrestrial MEC networks due to high-speed motion. Also, the workloads of satellite networks are more variable and uneven than terrestrial edge clouds since constellations cover the entire earth's surface.

Computation peer offloading among multiple satellites faces several challenges: 1) Traffic arrivals in satellite networks vary with geographical position and time zone. The limited computing resources among multiple satellites should be jointly leveraged to provide satisfactory computation service all the time. 2) The power supply of satellites is restricted as the solar panels are not always oriented towards sunlight, and the battery cells have a finite cycle life [32]. Peer offloading must consider energy consumption to avoid the continuous overwork of a single satellite that will drain the energy and cause it to age quickly. 3) Reduced computation delay is achieved by offloading tasks to light-loaded satellites multiple hops away but at the expense of transmission

overheads, so computing cooperation requires jointly optimizing the communication and computation overheads.

In this paper, we propose a multi-hop computation peer offloading scheme in MEC-enabled satellite networks. We initiate the offloading scheduling on the network side, and the access satellite offloads tasks to cooperative satellites via inter-satellite links (ISLs). Compared to single satellite computing, users are unaware of offloading schedules and can offload tasks to satellites out of sight in the proposed scheme. More tasks are served on the satellite edge computing platform, so the on-board resources are better utilized, and the propagation delay and congestion of downlinks are further alleviated. Besides, the proposed scheme can be applied to a broader range of services, from terrestrial network services to space-native data processing.

Specifically, we formulate the problem as maximizing the utilization of computing and communication resources while minimizing energy consumption. To reduce the complexity of the interplay among communication, computation, and energy consumption, we develop a multi-hop computation offloading model that incorporates satellites' computing and forwarding behaviors. Besides, due to the unfeasibility of knowing complete traffic arrival information, we leverage the online Lyapunov framework to optimize long-term performance. The delayed online learning technique is leveraged to help make decisions with the uncertainty of future workloads. The prediction loss is proved to be upper bounded.

The main contributions are summarized as follows:

- We design a satellite peer offloading scheme, where offloading is performed along multi-hop paths to explore collaborative computing capabilities. The proposed scheme efficiently balances the uneven workloads and improves the resource utilization of satellite networks.
- We formulate a Multi-Hop Satellite Peer Offloading (MH-SPO) problem as a global optimization problem, which jointly optimizes the latency and energy consumption under system backlogs and resource constraints. The Lyapunov framework is introduced to transform MH-SPO into an online per-slot optimization problem.
- We propose an online distributed decision-making scheme to solve MH-SPO. By gap-preserving reduction, we transform the network-wide Lyapunov-based cost minimization problem into several sub-problems conducted on individual satellites. The proposed algorithm can achieve close-to-optimal performance.
- We conduct simulations to evaluate the performance of the proposed scheme and compare it with baseline solutions. Simulation results show that the proposed scheme significantly improves the system performance.

The rest of this paper is organized as follows. Section II reviews related works. The system model is presented in Section III. The problem formulation and proposed solution are presented in Section IV. Simulations are carried out in Section V, followed by the conclusion in Section VI.

II. RELATED WORKS

In this section, we first study the MEC-enabled satellite networks and the fundamental offloading problem, and then review

how load balancing is tackled in the terrestrial MEC networks and explain why they do not apply to satellite networks.

A. MEC-Enhanced Satellite Networks

Satellite edge computing brings computing from "anytime" to "anywhere". In recent studies on the integration of MEC and satellite networks, satellites have evolved from being solely used for transmission [33], [34], [35] to directly processing computation tasks, leading to the emergence of in-orbit computing.

The advancements in satellite design have paved the way for in-orbit computing. For instance, improved power generation and storage technologies make in-orbit computing less energy-intensive [10]. Satellites equipped with advanced computation servers [11] and laser communication terminals [36], [37], [38] are capable of delivering high-performance computation and transmission. Furthermore, the feasibility of in-orbit computing is further enhanced by the fact that a significant portion of satellite network infrastructure is idle at any given moment due to the dispersed population. In [11], the authors explored the opportunities and challenges of in-orbit computing both qualitatively and quantitatively, and outlined several potential applications.

Several brand-new architectures for in-orbit computing have been proposed. [5] proposed the architecture of satellite edge computing and several significant technical challenges. The authors in [9] presented a SDN/NFV-based architecture of the MEC-enabled satellite-terrestrial integrated network (STIN) and explained critical functional parts. The authors emphasized that task scheduling should be decentralized in the MEC-enabled satellite networks, because the broad coverage and enormous scale of constellations would result in inefficiency by centralized control. This idea is adopted in this work. Besides, [39], [40] proposed utilizing GEO satellites to offload computation workloads from LEO satellites. The authors in [8] proposed a two-layer computing architecture consisting of terrestrial data center and satellite fog computing. They also designed the hardware and software deployments on the satellite computing platforms. A cross-layer cooperative computing scheme between space/air/ground layers was shown in [6]. The authors in [10] customized a computing system for camera-equipped nanosatellite constellations to process the sensed data locally.

In this paper, we adopt LEO satellites for computation offloading due to LEO's ultra-low terrestrial-satellite delay, high-capacity inter-satellite bandwidth, low design and manufacturing cost, and large-scale launches. Based on the architectures above, we present detailed computation offloading schemes in MEC satellite networks in the following.

B. Computation Offloading in MEC Satellite Networks

In the MEC-enhanced satellite networks, computing offloading is a fundamental problem. Many recent works investigated offloading strategies in MEC STINs. Various optimization goals included single objects (e.g., service latency [16], user energy consumption [14], [15]) and multiple objects (e.g., task processing delay and user energy consumption [7], service latency and users' satisfaction proportion [17]). However, these works initiated offloading schedules from the user side, ignoring the

capacities of satellites. In addition to the limited computation and communication resources, the MEC-enabled satellites are also energy-constrained [32]. Satellites are powered by solar panels and battery cells. The power supply may not be persistent because the solar panels used for electricity generation are not always oriented towards sunlight. The battery cells for energy storage have a finite cycle life. Therefore, it is essential to schedule the offloading strategies from the network's perspective and consider the satellites' energy consumption.

The authors in [13] considered both the satellites' computation energy consumption and the response time of tasks. Besides, in [18], each user device may selfishly choose the best offload target in a game-theoretic framework, according to satellite computation energy consumption, user transmission energy consumption, and task response delay. Nevertheless, most of these two papers, together with [7], [14], [15], [16], [17], adopted the single-satellite offloading strategy while neglecting the cooperation among peer satellites. Due to the constrained offloading strategy, the requests queued in high-load satellite nodes will suffer long delays. The idle resources in sparsely populated areas are wasted. A large amount of raw data will be forwarded to the ground data center because of the constrained processing capability of single satellites, which will cause the congestion of downlinks.

Therefore, we initiate the satellite peer computation offloading scheme from the network side and jointly consider the latency and energy consumption under the constraints of system backlogs and resources. The closest work to the proposed scheme is [41], in which an access satellite could distribute a task to four neighbor satellites. However, this one-hop offload scheme still can not fully utilize the system resources and achieve efficient task processing. Hence, in this paper, we adopt the multi-hop offloading scheme.

C. Computation Offloading in Terrestrial MEC Networks

Computation offloading has been widely studied in terrestrial MEC networks. For example, in [30], the authors tackled computation offloading in NOMA-assisted multi-access MEC by jointly optimizing the mobile users' offloaded workloads and the NOMA transmission-time allocation, with a focus on the offloading scheduling between users and edge servers. In this paper, however, our focus shifts to the offloading among satellite edge nodes. In satellite-terrestrial networks, a mobile user usually has few access satellite options because the access policy is restricted by connection time, channel quality, and so on. Therefore, more emphasis must be placed on offloading among satellites initiated by the access satellites to minimize overall delay for the users.

Peer computation offloading has also been studied in previous works. Load balancing is a key aspect during the peer offloading process. Spatially uneven computation workloads are a common issue in densely deployed small-cell networks. In [31], the authors addressed this problem by grouping MEC servers and controlling task offloading in server clusters. In [21], the authors developed an online peer offloading framework among small-cell base stations to reduce computation latency at overloaded base stations and ensure high-quality service for end

TABLE I
MAJOR NOTATIONS

Notation	Description
N, P, S, F	constellation parameters: the number of satellites, planes, and satellites per plane, constellation phasing factor
$\mathcal{N}, n$	set and index of satellites
N_n, m	set and index of satellite n 's neighbors
$\mathcal{T}, t$	set and index of time slots
(n, m)	index of physical links
c_n	computation service rate of n
$b_{nm}(t)$	transmission capacity of (n, m) in t
$T_{nm}(t)$	propagation delay of (n, m) in t
$\mathcal{I}, I(t)$	set of all active users and set of active users in t
$I_n(t)$	set of active users in n 's coverage in t
s^i	task i 's size
h^i	task i 's required number of cycles for processing one bit
t_0^i, d_{max}^i	task i 's arrival slot and deadline
$a_n^i(t)$	task i 's admission decision on n in t
$\alpha_n(t)$	arrival rate on n in t
$\mathcal{Q}_n^F(t)$	set of task requests waiting for offloading decisions in t
$\mathcal{Q}_n^B(t)$	set of requests for tasks to be computed in t
$Q_n^F(t)$	workload amount of forwarding on n in t
$Q_n^B(t)$	workload amount of computing on n in t
$\alpha_n^i(t)$	task i 's local computing decision on n in t
$\alpha_n(t)$	amount of n 's local computing workloads in t
$\beta_{nm}^i(t)$	task i 's offloading decision from n to m in t
$\beta_{nm}(t)$	amount of offloading workloads from n to m in t
$\beta_n(t)$	amount of offloading workloads out of n in t
p^i, n_k^i	task i 's offloading path and intermediate node
$r_n^i(t)$	task i 's processed workloads on n in t
$d_n^{i,C}, e_n^{i,C}$	computation delay and energy consumption of i on n
P_t	transmission power
$d^{i,T}$	offloading delay of i
$e^{i,T}$	transmission energy consumption of i
ρ_d, ρ_e	weighting parameters of delay and energy consumption
E_n, B_{nm}	local computing capacity, offloading capacity

users. However, these approaches do not apply to load balancing in satellite MEC networks, as the mobility patterns in satellite networks differ greatly from those in terrestrial mobile networks. With satellites orbiting the Earth periodically, the length and connectivity of inter-satellite links (ISLs) vary and the channel quality of ISLs change, requiring consideration each time an offloading decision is made. In this paper, we propose a peer offloading scheme suitable for satellite dynamics.

III. SYSTEM MODEL

In this section, we first introduce the network model. Then, we present the design of satellite peer computation offloading. Finally, the computation model and transmission model are defined. The major notations of the key metrics are presented in Table I.

A. Network Model

The multi-hop satellite peer offloading scheme is shown as 1. We consider a Walker Constellation. N is the total number of satellites in a constellation which is composed of P orbit planes, each consisting of S satellites. F is the constellation phasing factor. The MEC-enabled satellite network topology $\mathcal{G} = (V, E)$ varies over time. V is the set of all satellite edge computing nodes, indexed by $\mathcal{N} = (1, \dots, N)$, and E represents the set of ISLs. Two nodes that can establish a physical connection are neighbors of each other, and the neighbor set for a node $n \in \mathcal{N}$ is N_n . The computing capability of satellite n is characterized by its computation service rate c_n (CPU frequency). The transmission capacity and propagation delay of link (n, m) , $m \in N_n$

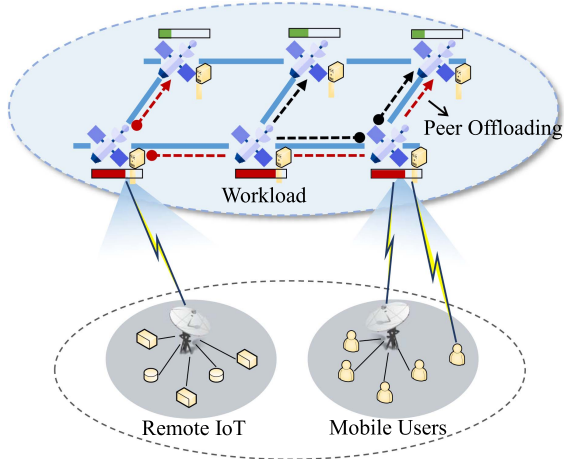


Fig. 1. MEC-enhanced multi-hop satellite peer offloading.

changes over time, which are defined as $b_{nm}(t)$ (bits per second) and $T_{nm}(t)$, respectively.

Here, we assume that users are allocated to the closest satellite or the satellite with the longest dwell time in line of sight, but the proposed scheme is also compatible with other access strategies. Let $\mathcal{I}$ be the set of all active users. $I(t)$ denotes the set of active users at time t , and the active user set in satellite n 's coverage is $I_n(t)$. Every user is assumed to access at most one satellite for simplicity, so $\sum_{n \in \mathcal{N}} I_n(t) = I(t) \subseteq \mathcal{I}$.¹

B. Multi-Hop Computation Peer Offloading

We adopt the idea of snapshots to model the dynamics of satellite networks. The topology information is assumed to be fixed in each snapshot but varies among different snapshots. The operational timeline is discretized into several snapshots, and each snapshot is divided into many time slots (we use t to denote time slots in the following). At the beginning of each time slot t , satellite edge nodes make task admission and offloading decisions to every received task request. In the rest of t , satellites perform task forwarding and processing according to the decisions. The set of all time slots is $\mathcal{T}$.

In each time slot, the computation peer offloading scheme of every satellite edge node includes three parts:

1) *Task Admission*: At the beginning of t , the satellite receives all task requests from the active users it covers. The task request corresponding to task i contains i 's information specified by a tuple $(s^i, h^i, t_0^i, d_{\max}^i)$, where $s^i \in [s^{\min}, s^{\max}]$ is the task size in bits, $h^i \in [h^{\min}, h^{\max}]$ is the required number of CPU cycles for processing one bit of the task, t_0^i is the time slot when task arrives its access satellite, and $d_{\max}^i$ is the deadline. The satellite makes admission decisions according to its current workload level, which means if the satellite workload is heavy, some task requests will be rejected. The rejected requests wait for task admission decisions in the next time slot or access another light-loaded satellite. Task i 's admission result at t is $a_n^i(t)$, which is 1 if the task is admitted and 0 otherwise. The total task arrival rate of satellite n at t is defined as $a_n(t) = \sum_{i \in I_n(t)} a_n^i(t) s^i$.

¹In the rest of this paper, we will use "user" and "task" interchangeably.

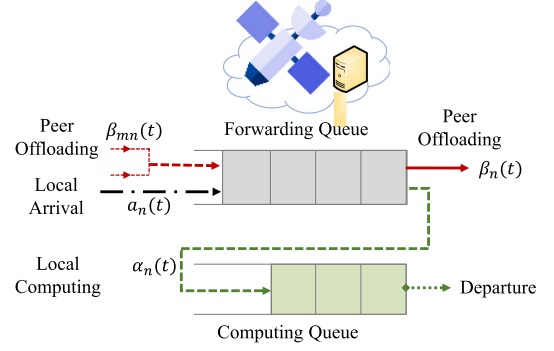


Fig. 2. Queue model for satellite peer offloading.

Since the sizes of task requests and admission results are usually small and constant, it is acceptable to assume that the interaction message can be transmitted immediately and the up-link from users to the satellite is not congested. Besides, the time for making task admission decisions can be ignored due to the low complexity of the algorithm (see Section IV-E). As for the rejected task requests, we model the response delay waiting for a successful admission decision as a statistical variable related to user distribution density, satellite network load, etc. Hence, the average response delay is the same for all task requests, and we exclude it from the total delay. A more precise model of response delay will be studied in future works.

2) *Task Scheduling*: This process happens at the beginning of t but after task admission. The satellite organizes the requests of successfully admitted tasks and the requests of tasks offloaded to itself for task scheduling decision, and determines whether each task will be processed locally or offloaded to one of the neighboring satellites. Satellite edge nodes share backlog and link quality information in every slot. As the size of interaction messages is small and the decisions can be made in polynomial time (see Section IV-E), this process will occupy a small portion of a time slot.

3) *Task Processing*: Based on the task scheduling results, satellite edge nodes will carry out the processing and transmitting of tasks, which consume the majority of the time slot.

As described above, satellites act as both task processors and intermediate forwarders in satellite computation peer offloading. To distinguish and formulate the forwarding and computing behaviors, we maintain two virtual task queues named forwarding queue Q^F and computing queue Q^B for each MEC-enabled satellite node. The queue model is demonstrated as Fig. 2. For satellite n , $Q_n^F(t)$ is the set of task requests waiting for offloading decisions, and $Q_n^B(t)$ is the set of requests for tasks to be computed during time slot t . Specifically, at the beginning of slot t , $Q_n^F(t)$ and $Q_n^B(t)$ are updated. The requests for successfully admitted tasks are added to $Q_n^F(t)$. Then, for every task request in $Q_n^F(t)$, satellite n decides whether to offload the task to neighbors or to process it locally. If the former happens, assuming task i will be sent from n to $m \in N_n$, so $\beta_{nm}^i(t) = 1$ (otherwise 0), then the request of i will join $Q_m^F(t+1)$ and wait for the next offloading decision. If the latter happens, $\alpha_n^i(t) = 1$ (otherwise 0), the request of i will join $Q_n^B(t+1)$ and stay in

Q_n^B until i is finished. We define $Q_n^F(t)$ and $Q_n^B(t)$ to capture the workload amount of forwarding and computing, respectively:

C. Computation Model

$$Q_n^F(t+1) = \max \{Q_n^F(t) - \alpha_n(t) - \beta_n(t), 0\} + a_n(t) + \sum_{m \in N_n} \beta_{mn}(t), \quad (1)$$

$$Q_n^B(t+1) = \max \left\{ Q_n^B(t) - \sum_{i \in Q_n^B(t)} r_n^i(t), 0 \right\} + \alpha_n(t). \quad (2)$$

Note that $\alpha_n(t) = \sum_{i \in Q_n^F(t)} s^i \alpha_n^i(t)$ represents the amount of locally processed tasks, $\beta_n(t) = \sum_{m \in N_n} \beta_{nm}(t) = \sum_{m \in N_n} \sum_{i \in Q_n^F(t)} s^i \beta_{nm}^i(t)$ is the total amount of tasks sent out by n , and $\beta_{nm}(t)$ specifies the tasks transmitted from n to m . Newly arrived tasks $a_n(t)$ and tasks offloaded from neighbors $\sum_{m \in N_n} \beta_{mn}(t)$ are admitted by n at the end of slot t . $r_n^i(t)$ is task i 's processed workloads at node n during slot t , explained in the next subsection.

We assume that a task can be forwarded several hops away but can only be processed on one MEC-enabled satellite node. The admitted task i 's offloading path is $p^i = \{n_k^i | k = 0, \dots, K\}$, which means task i that initially accesses satellite n_0^i is forwarded multiple hops away, and is finally processed on the satellite n_K^i .

The MEC-enabled satellites process the tasks queued in Q^B . We adopt the fair-share scheduling of computing resources inspired by the Rate Control Protocol (RCP) [25]. Every computing node assigns the same rate to all tasks and updates the rate allocation every slot. Compared to the priority-based scheduling, fair-share computing resource scheduling guarantees that all tasks share the same resources without preemption. Thus, for any task i whose request is buffered in $Q_n^B(t)$, the allocated computing frequency is

$$f_n^i(t) = \frac{c_n}{|Q_n^B(t)|}, \quad (3)$$

where c_n is the CPU frequency of satellite n (CPU cycles per second) and $|Q_n^B(t)|$ is the number of tasks locally processed.

For task i , the amount of workloads processed at slot t is described as

$$r_n^i(t) = \min \left\{ \frac{c_n}{h^i |Q_n^B(t)|}, s^i - \sum_{\tau=0}^{t-1} r_n^i(\tau) \right\}, \quad (4)$$

where the second term in the curly bracket is the residual workloads of i at the beginning of slot τ . Tasks will leave the system as soon as they are completely served, which means the residual workloads are 0. We assume that once the satellite begins to process a task, it will continuously work on it until it is finished. We assume that task i starts being processed at satellite n at slot t , which means $\alpha_n^i(t) = 1$. The *computation delay* of task i is defined as

$$d^{i,C} = \Delta t, \quad (5)$$

where Δt is calculated according to $\sum_{\tau=t+1}^{t+\Delta t} r_n^i(\tau) = s^i$ if $\alpha_n^i(t) = 1$.

Also, if $\alpha_n^i(t) = 1$, the *computation energy consumption* of satellite n for processing task i is

$$e^{i,C} = \sum_{\tau=t+1}^{t+\Delta t} \kappa r_n^i(\tau) f_n^{i^2}(\tau), \quad (6)$$

where κ is a coefficient depending on the chip architecture.

D. Transmission Model

Transmissions occur on the uplinks from user devices to satellites, the wireless links among satellites, and the downlinks from satellites to the ground. The length of uplinks is approximately the same, and this part of transmission does not consume the satellite energy, so we do not consider the uplink transmission part. The sizes of computation results are small, so we ignore the transmission delay and energy consumption from the computing satellite to the ground. Here, we only consider the transmission of wireless links among satellites.

We adopt the laser ISLs as the inter-satellite wireless communication technology. Optical wireless communication takes advantages of high bandwidth, unlicensed spectrum allocation, reduced antenna size, and improved channel security compared with conventional radio frequency techniques. It has been considered a promising paradigm for future inter-satellite communications.² According to [45], the transmission between two satellites faces challenges of doppler shift, acquisition and tracking, and background radiations.

1) *Doppler Shift*: Doppler shift happens to a pair of satellite carriers moving with different relative velocity. The destination satellite will shift its central wavelength, and thus a small portion of wavelength leaks to adjacent channels, which results in loss of data. The normalized Doppler shift between two adjacent satellites in the same orbit without any compensation methods has the following form [46]:

$$\frac{\Delta\lambda}{\lambda_s} = -\frac{\omega R}{c} \cdot \frac{a \sin(2\omega t + 2\pi m)}{[a \cos(2\omega t + 2\pi m) + b]^{\frac{1}{2}}},$$

where

$$a = \sin^2 \theta \left[1 - \cos \left(\frac{2\pi}{P} \right) \right]$$

$$b = 2 + 2 \cos \theta \sin \left(\frac{2\pi}{P} \right) \sin(2\pi m)$$

$$- \left[\sin^2 \theta - \sin^2 \theta \cos \left(\frac{2\pi}{P} \right) + 2 \cos \left(\frac{2\pi}{P} \right) \right] \cos(2\pi m)$$

$$m = \frac{F}{PS} + \frac{\omega T}{\pi}.$$

Here, $\Delta\lambda/\lambda_s$ is the normalized Doppler shift, ω is the angular velocity of the satellite, R is the distance between the satellite

²The high-speed laser ISLs have been successfully demonstrated by the European space agency [42], NASA [43] and many other agencies. SpaceX has launched Starlink satellites equipped with laser communication terminals on January 24, 2021, and Telesat's Lightspeed laser ISLs will start offering services in 2023 [44]. Hence, the assumption of laser ISLs is entirely reasonable in this paper.

and the earth center, t is the time period, c is the speed of light, P is the number of orbit planes, F is the constellation phasing factor, S is the number of satellites per plane, and T is the propagation delay of two satellites varying over time, which is defined in Section III-A. Moreover, the normalized Doppler shift over two adjacent satellites in adjacent orbits without any compensation methods is presented as [46]

$$\frac{\Delta\lambda}{\lambda_s} = \frac{R\dot{\varphi}(t+T)}{c} \cdot \frac{\sin[\varphi(t+T) - \varphi(t_0)] \cos\phi(t_0)}{\sqrt{2 - 2\cos[\varphi(t+T) - \varphi(t_0)] \cos\phi(t_0)}},$$

where $\dot{\varphi}(t+T)$ is the angular velocity of the destination satellite, t_0 is the time when two satellites have the minimum distance, $\varphi(t+T) - \varphi(t_0)$ is the angular distance between the destination satellite and position where two satellites have minimum distance, $\phi(t_0)$ is the angular distance when two satellites have minimum distance.

Although the representation of the Doppler shift is complex, it is easy to predict and calculate once the constellation parameters are given. The parameters are fixed when designing constellations, and the velocity and position of any satellite at any time are known in advance, so the Doppler shift between any pair of satellites during any period can be predicted and compensated by efficient methods. Common methods include optical phase-lock loop, optical injection locking, optical injection phase-lock loop, etc [45]. Therefore, we suppose that the receiver of MEC-enhanced satellites has equipped with Doppler shift frequency compensators, and we exclude the effect of Doppler shift on ISL transmission.

2) *Acquisition and Tracking*: Satellites are commonly equipped with acquisition, tracking and pointing (ATP) systems to ensure the optical beam reaches the receiver reliably. However, the ATP system's vibration degrades the optical transmission link's performance by introducing a stochastic tracking noise created by the electro-optic tracker. Hence, we introduce the tracking noise N' and show its effect in the next point.

3) *Background Radiations*: The noise sources depend on the detection techniques and generally include detector (bulk dark current), receiver amplifier (pre-amplifier noise and thermal noise), shot noise, stellar and celestial radiant fluxes, and so on. We denote N_0 to represent this part of noise.

Considering the impact of the above three factors on ISL transmission, we express the data rate of link (n, m) at slot t as follows [47]:

$$b_{nm}(t) = \frac{P_t G_{tr} G_{re} L_f^t}{k T_s \cdot [E_b / (N' + N_0)]_{req} \cdot M}. \quad (7)$$

Wherein, the free space loss L_f^t can be shown as

$$L_f^t = \left(\frac{c}{4\pi \cdot SR(t) \cdot f} \right)^2.$$

Here, P_t is the constant transmission power of satellites. G_{tr} and G_{re} are the transmitting antenna gain and the receiving antenna gain, respectively. k is the Boltzmann's constant and T_s is the total system noise temperature. Besides, $E_b / (N' + N_0)$ is the required ratio of received energy-per-bit to certain noise-density, M is the link margin, and $SR(t)$ is the slant range.

The total offloading delay includes the transmission delay and the propagation delay. Assume task i 's offloading path is $p^i = \{n_k^i | k = 0, \dots, K\}$. Hence, the total offloading delay is

$$d^{i,T} = \sum_{t \in T} \sum_{k=0}^{K-1} \beta_{n_k^i n_{k+1}^i}(t) \left(\frac{s^i}{b_{n_k^i n_{k+1}^i}(t)} + T_{n_k^i n_{k+1}^i}(t) \right), \quad (8)$$

where $\beta_{n_k^i n_{k+1}^i}(t) = 1$ if the task i is offloaded from n_k^i to n_{k+1}^i along p^i during slot t , $b_{n_k^i n_{k+1}^i}(t)$ and $T_{n_k^i n_{k+1}^i}(t)$ are the transmission capacity and propagation delay of one hop along p^i . Suppose each satellite operates at a fixed transmission power P_t , the transmission energy consumption for sending task i is

$$e^{i,T} = \sum_{t \in T} \sum_{k=0}^{K-1} \beta_{n_k^i n_{k+1}^i}(t) \frac{s^i}{b_{n_k^i n_{k+1}^i}(t)} P_t. \quad (9)$$

IV. PROBLEM FORMULATION AND SOLUTION

In this section, we first formulate MHSPO as an offline global optimization problem. Then, we leverage the Lyapunov technique to develop a novel framework for making online per-slot computation peer offloading decisions with only current information. In order to alleviate the current perspective's limitations, we leverage the delayed online learning to predict the task's processing delay and energy consumption. The prediction loss is proved to be upper bounded. Finally, we propose an online queue-based distributed offloading control scheme to solve the problem. The performance of the solution is analyzed.

A. Problem Formulation

The computation offloading overhead of task i is defines as

$$C^i = \rho_d(d^{i,C} + d^{i,T}) + \rho_e(e^{i,C} + e^{i,T}),$$

where ρ_d, ρ_e denote the weighting overhead parameters of delay and energy consumption, respectively. We aim to make low-overhead computation offloading decisions for all tasks $i \in \mathcal{I}$, which means the optimization goal is $\min \sum_{i \in \mathcal{I}} C^i$. According to (5), (6), (8), and (9), C^i is relative to the system's computation offloading decisions over time. Hence, the optimization goal can be shown in another way:

$$\min \sum_{t \in T} C(t), \quad (10)$$

where $C(t)$ is defined as the system cost at slot t and can be represented as

$$\begin{aligned} C(t) = & \sum_{n \in \mathcal{N}} \sum_{i \in \mathbf{Q}_n^F(t)} \left(\alpha_n^i(t) [\rho_d d_n^{i,C} + \rho_e e_n^{i,C}] \right. \\ & + \sum_{m \in N_n} \beta_{nm}^i(t) \\ & \times \left[\rho_d \frac{s^i}{b_{nm}(t)} + \rho_d T_{nm}(t) + \rho_e \frac{s^i}{b_{nm}(t)} P_t \right] \Big). \quad (11) \end{aligned}$$

Here, $C(t)$ is the weighted sum of system delay and energy consumption corresponding to the offloading and computing

decision of every task in $\mathbf{Q}_n^F(t)$. If the system decides $\alpha_n^i(t) = 1$, it means task i will start being computed, so the computation delay (5) and energy consumption (6) must be added to the system cost at slot t . We replace $\beta_{n_k^i n_{k+1}^i}^i(t)$ in (8) and (9) with $\beta_{nm}^i(t)$ as $\beta_{nm}^i(t)$ can better display the system's offloading decisions. The $b_{n_k^i n_{k+1}^i}^i(t)$ and $T_{n_k^i n_{k+1}^i}^i(t)$ in (8) and (9) are also replaced with $b_{nm}(t)$ and $T_{nm}(t)$ for the same reason.

Formally, a satellite peer computation offloading mechanism determines the admission decisions $\mathbf{a} = \{\mathbf{a}(t) | \forall t \in \mathcal{T}\} = \{a_n^i(t) | \forall t \in \mathcal{T}, i \in I(t), n \in \mathcal{N}\}$ and the offloading decisions $\mathbf{s} = \{\mathbf{s}(t) | \forall t \in \mathcal{T}\} = \{s_n^i(t) | \forall t \in \mathcal{T}, i \in \mathbf{Q}_n^F(t), n \in \mathcal{N}\} = \{\alpha_n^i(t), \beta_{nm}^i(t) | \forall t \in \mathcal{T}, i \in \mathbf{Q}_n^F(t), m \in N_n, n \in \mathcal{N}\}$ to minimize the long-term system overhead. The MHSP0 problem is formulated as

$$\mathbf{P1} \quad \min_{\mathbf{a}, \mathbf{s}} \quad \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} C(t) \quad (12)$$

$$\text{s.t.} \quad \alpha_n^i(t) + \sum_{m \in N_n} \beta_{nm}^i(t) \leq 1, \quad \forall i \in \mathbf{Q}_n^F(t), n \in \mathcal{N} \quad (12a)$$

$$d_m^{i,C} + \frac{s^i}{b_{nm}(t)} + T_{nm}(t) \leq d_n^i, \quad \forall i \in \mathbf{Q}_n^F(t), \exists \beta_{nm}^i(t) = 1, m \in N_n \quad (12b)$$

$$\sum_{i \in \mathbf{Q}_n^F(t)} \alpha_n^i(t) \leq E_n, \quad \forall n \in \mathcal{N} \quad (12c)$$

$$\sum_{i \in \mathbf{Q}_n^F(t)} s^i \beta_{nm}^i(t) \leq B_{nm}, \quad \forall m \in N_n, n \in \mathcal{N} \quad (12d)$$

$$\lim_{|\mathcal{T}| \rightarrow \infty} \frac{1}{|\mathcal{T}|} \sum_{t=0}^{|\mathcal{T}|-1} \sum_{n \in \mathcal{N}} \mathbb{E} \{Q_n^F(t) + Q_n^B(t)\} < \infty, \quad \forall n \in \mathcal{N}. \quad (12e)$$

Constraint (12a) requires that for a task i , satellite n can locally process it, offload it to one of its neighbors, or keep its request waiting in queue $\mathbf{Q}_n^F$. An offloading strategy from n to m is beneficial if it satisfies constraint (12b). The forwarding behavior happens only when offloading to m does not incur higher completion delay than n 's local computing. Constraints (12c) and (12d) are scheduling capacity constraints. E_n is the maximum number of tasks dispatched to n 's computing queue at one slot for guaranteed processing rate, and $B_{nm} \in [0, B^{\max}]$ specifies the limited offloading capacity, which is relative to bandwidth and the length of slots. Constraint (12e) is the queue stability constraint, which means a bounded time-average backlog. Due to the limited satellite resources, the amount of workloads entering the network is restricted to guarantee the service rate of admitted tasks.

The major challenge that impedes the derivation of the optimal solution to **P1** is the lack of future information. Optimally solving **P1** requires complete task arrival information across all time slots, which is difficult to get in advance. Moreover, even though the task arrival information is available, the solution

complexity is unaffordable. The scale of the solution searching space is exponential to the number of tasks, the timeline range, and the satellite network topology scale. It is impossible to derive the optimal solution in finite time. These challenges call for an online efficient optimization approach to perform satellite peer offloading.

B. Lyapunov Optimization Based Online Framework

Lyapunov optimization has been extensively used to achieve optimal control in dynamic systems. In **P1**, the long-term optimization goal (12) and the time-average queue stability constraint (12e) couple per-slot offloading decisions $(\mathbf{a}(t), \mathbf{s}(t))$ across time slots. To address this challenge, we adopt the Lyapunov drift-plus-penalty framework [48] for online joint network stability and system overhead minimization. Several definitions are given below.

Definition 1: Let $\Theta(t) = [\mathbf{Q}^F(t), \mathbf{Q}^B(t)]$ be the aggregate queue vector, where $\mathbf{Q}^F(t) = \{Q_n^F(t) | \forall n \in \mathcal{N}\}$ and $\mathbf{Q}^B(t) = \{Q_n^B(t) | \forall n \in \mathcal{N}\}$. We define the perturbed Lyapunov function as

$$L(\Theta(t)) = \frac{1}{2} \|\mathbf{Q}^F(t) - \boldsymbol{\theta}\|^2 + \frac{1}{2} \|\mathbf{Q}^B(t)\|^2, \quad (13)$$

where $\boldsymbol{\theta} = \theta_n \cdot \mathbf{1}^N$ with θ_n being perturbation parameters.

$L(\Theta(t))$ represents a scalar metric of the queue length in all queues. A small value of $L(\Theta(t))$ implies that all the queue backlogs are small, which means the virtual queues are not prone to overflow and have strong stability. $Q_n^F(t)$ converges towards θ_n instead of zero, which ensures each satellite node has a certain amount of workloads to schedule, avoiding the waste of communication and computation resources.

Definition 2: For system stability, we define *one-slot Lyapunov drift* as

$$\Delta(\Theta(t)) = \mathbb{E} \{L(\Theta(t+1)) - L(\Theta(t)) | \Theta(t)\}. \quad (14)$$

$\Delta(\Theta(t))$ captures expected changes of the backlogs over slots. With the designed Lyapunov drift, the long-term backlog constraint (12e) is enforced if $\Delta(\Theta(t))$ converges towards zero, which means the queues are stable.

Definition 3: For optimizing the system performance, we define the instant system overhead $C(t)$ as *one-slot Lyapunov penalty*, which is the weight sum of latency and energy consumption in (11).

Now, we present the online approach for solving **P1** in the framework of Lyapunov drift-plus-penalty. In each time slot t , network operators determines the computation peer offloading strategies by solving **P2** formulated below

$$\mathbf{P2} \quad \min_{\mathbf{a}(t), \mathbf{s}(t)} \quad \Delta(\Theta(t)) + V \cdot \mathbb{E} \{C(t) | \Theta(t)\} \quad (15)$$

$$\text{s.t.} \quad (12a), (12b), (12c), (12d).$$

The first term in **P2** is added aiming to satisfy the long-term backlog constraint (12e) in an online manner and the second term is to minimize the system overhead. The non-negative weight V is chosen to affect a performance tradeoff between queue stability and system overhead. To give a brief explanation, by

Algorithm 1: Prediction Policy on Node n at Slot t .

```

1:  $\eta_n \leftarrow \frac{Q_n^{B,\max}}{\sqrt{T+D}}$ ;
2: Observe the actual current workload  $Q_n^B(t)$ ;
3: for each slot  $\tau \in T_t$  do
4:   Derive the predicted workload  $Q_n^{B*}(\tau + 1)$  by (18);
5: end for
6: Share the future workload information with neighbors
  in  $N_n$  and receive neighbors' workload information;
7: for each task  $i \in Q_n^F(t)$  do
8:   for each node  $m \in N_n \cup \{n\}$  do
9:      $s' \leftarrow s^i, t' \leftarrow t, e' = 0$ ;
10:    while  $s' > 0$  do
11:       $f_m^i(t') \leftarrow \frac{c_n}{|Q_n^{B*}(t')|}$ ;
12:       $r_m^i(t') \leftarrow \min\{\frac{f_m^i(t')}{h^i}, s'\}$ ;
13:       $e' \leftarrow e' + \kappa r_m^i(t') f_m^i(t'), s' \leftarrow s' - r_m^i(t'),$ 
         $t' \leftarrow t' + 1$ ;
14:    end while
15:     $d_m^{i,C} \leftarrow t' - t, e_m^{i,C} \leftarrow e'$ ;
16:  end for
17: end for

```

considering the additional first term in **P2**, the network operator takes into account the system stability in current-slot decision-making. When $\Delta\Theta(t)$ is larger, minimizing the queue length is more critical for the network operator, and hence the long-term system stability is satisfied in the long run without knowing the future information in advance. If $\Delta\Theta(t)$ stays low, minimizing the system overhead is the main goal.

For any feasible solution of **P2**, we have (16) shown at the bottom of this page, where $\tilde{Q}_n^F(t) = Q_n^F(t) - \theta_n, B_1 = \frac{1}{2} \sum_{n \in \mathcal{N}} \{[s'E_n + (N-1)B^{\max}]^2 + [s'I^{\max} + (N-1)B^{\max}]^2 + c_n^2 + (s'E_n)^2\}$, $B_2(t) = -\sum_{n \in \mathcal{N}} \sum_{i \in Q_n^B(t)} Q_n^B(t) r_n^i(t)$, which is a known constant at the beginning of slot t . The details are given in Appendix A, available online. Notice that solving **P2** is exactly to minimize the right hand side of (16).

By solving the per-slot minimization problem of **P2**, we persistently approach the optimal offline solution of **P1** in a greedy manner. The rigorous performance bound of **P2** compared to **P1** is analyzed in Section IV-E.

C. Learning-Based Prediction Algorithm

Since the values of $d^{i,C}$ and $e^{i,C}$ in **P2** are not precisely known at the decision slot, we propose a workload prediction policy based on delayed online learning to encourage high-quality decision-making.

Once a task i is assigned to node n and starts to be computed at slot t , the $d^{i,C}$ and $e^{i,C}$ highly rely on the workload levels in the following slots (i.e., $Q_n^B(t+1), Q_n^B(t+2), \dots$) until the slot when i is finished. We define a non-negative integer prediction window length d_t [25], [49] for every t , which means the prediction window is $T_t = \{t+1, \dots, t+d_t\}$. To accommodate user diverse demands in deadline, we make $d_t = d_{\max} = \max_{i \in \mathcal{I}} d_{\max}^i$ for every slot t .

Let $Q_n^B(t)$ be the actual computing workload amount of MEC-enabled satellite node n at slot t , and $Q_n^{B*}(t) = \{Q_n^{B*}(\tau) | \forall \tau \in T_t\}$ be the predicted workloads. We seek to minimize the prediction error (a.k.a. the loss function), $f_{n\tau}(Q_n^{B*}(\tau)) = |Q_n^{B*}(\tau) - Q_n^B(\tau)|$, $\forall n \in \mathcal{N}, \tau \in T_t$, to bound the sub-optimality gap due to imperfect prediction. $f_{n\tau}$ is a convex function on $[0, Q_n^{B,\max}]$, where $Q_n^{B,\max} = \max_{t \in T} Q_n^B(t)$. A loss minimization problem is constructed as follows:

$$Q_n^{B*}(\tau) \in [0, Q_n^{B,\max}] \sum_{t \in T} \sum_{\tau \in T_t} f_{n\tau}(Q_n^{B*}(\tau)). \quad (17)$$

We use the Delayed Online Gradient Descent (DOGD) method [50], [51] to compute $Q_n^{B*}(\tau)$ that minimizes $\sum_{\tau \in T_t} f_{n\tau}(Q_n^{B*}(\tau))$ for each time slot t . Specifically, without the knowledge of the actual $Q_n^B(\tau)$ for $\tau \in T_t$, $f_{n\tau}(Q_n^{B*}(\tau))$ is not available. Hence, it is based on the feedback observed from t to make the workload predictions $Q_n^{B*}(\tau)$ for every future slot $\tau \in T_t$. The update rule for each slot $\tau \in T_t$ is as follows:

$$Q_n^{B*}(\tau + 1) = Q_n^{B*}(\tau) - \eta_n \nabla f_{nt} |_{Q_n^{B*}(\tau)}. \quad (18)$$

The essence of (17) is to iterate the prediction in the negative direction of the gradient of the loss received at the current slot.

The step size η_n is set as $\frac{Q_n^{B,\max}}{\sqrt{T+D}}$ with $D = \sum_{t \in T} \sum_{\tau \in T_t} d_\tau = \frac{1}{2} T d_{\max} [1 + d_{\max}]$ denoting the sum of delays over all slots. $\nabla f_{nt} |_{Q_n^{B*}(\tau)}$ equals 1 if $Q_n^{B*}(\tau) > Q_n^B(\tau)$; -1 if $Q_n^{B*}(\tau) < Q_n^B(\tau)$; 0, otherwise.

The learning-based prediction policy and the derivation process of $d^{i,C}$ and $e^{i,C}$ are shown in Algorithm 1.

$$\Delta(\Theta(t)) + V \cdot \mathbb{E} \{C(t) | \Theta(t)\}$$

$$\begin{aligned}
&\leq \sum_{n \in \mathcal{N}} \mathbb{E} \left\{ \sum_{i \in I_n(t)} \alpha_n^i(t) \tilde{Q}_n^F(t) | \Theta(t) \right\} + \sum_{n \in \mathcal{N}} \mathbb{E} \left\{ \sum_{i \in Q_n^F(t)} \alpha_n^i(t) \left[s^i Q_n^B(t) - s^i \tilde{Q}_n^F(t) + V \rho_d d_n^{i,C} + V \rho_e e_n^{i,C} \right] | \Theta(t) \right\} \\
&+ B_1 + B_2(t) \\
&+ \sum_{n \in \mathcal{N}} \sum_{m \in N_n} \mathbb{E} \left\{ \sum_{i \in Q_n^F(t)} \beta_{mn}^i(t) s^i \tilde{Q}_n^F(t) - \sum_{i \in Q_n^F(t)} \beta_{nm}^i(t) \left[s^i \tilde{Q}_n^F(t) - V \rho_d \left(\frac{s^i}{b_{nm}(t)} + T_{nm}(t) \right) - V \rho_e \frac{s^i}{b_{nm}(t)} P_t \right] | \Theta(t) \right\}. \quad (16)
\end{aligned}$$

We analyze the performance of the proposed prediction strategy, which is defined as *regret*. Let $\mathbf{Q}_n^{B'}(t) = \{Q_n^{B'}(\tau) | \forall \tau \in \mathbf{T}_t\}$ be the best static predictor in hindsight obtained by the strategy in [52] with full knowledge of workloads. We have

$$\text{Regret}_n^T(\text{DOGD}) = \sum_{t \in T} \sum_{\tau \in \mathbf{T}_t} \left[f_{n\tau}(Q_n^{B*}(\tau)) - f_{n\tau}(Q_n^{B'}(\tau)) \right]. \quad (19)$$

Lemma 1: The overall regret of Algorithm 1 is upper bounded compared with the best static prediction strategy.

$$\begin{aligned} \text{Regret}^T(\text{DOGD}) &= \sum_{n \in \mathcal{N}} \text{Regret}_n^T(\text{DOGD}) \\ &\leq \sum_{n \in \mathcal{N}} \frac{d_{\max}}{2\eta_n} + \eta_n \left(\frac{T d_{\max}}{2} + 2D \right). \end{aligned} \quad (20)$$

The proof of (20) can be found in [25].

D. Distributed Solution

The centralized solution of **P2** is not feasible in the MEC-enhanced satellite networks. The large network scale and high topology dynamism bring huge complexity of centralized computation. It is also not efficient for a centralized controller to distribute offloading decisions. Therefore, the task scheduling should be carried out directly on satellite computing nodes [9]. A distributed offloading decision scheme is needed.

First, **P2** should be decoupled for individual satellites. Note that in (15), the offload workloads are counted bidirectionally, which means $\beta_{nm}^i(t)$ and $\beta_{mn}^i(t)$ coexist in the third term of (16). But a node n can not acquire neighbors' $\beta_{mn}^i(t)$ until the next slot $t + 1$. To solve the problem, we leverage the gap-preserving techniques to transform (16) into a single direction optimization problem. Given $\sum_{i \in \mathbf{Q}_n^F(t)} s^i \beta_{nm}^i(t) + \sum_{i \in \mathbf{Q}_m^F(t)} s^i \beta_{mn}^i(t) \leq B_{nm} + B_{mn}$, we have the following new problem:

$$\begin{aligned} \mathbf{P3} \quad & \min_{\mathbf{a}(t), \mathbf{s}(t)} \sum_{n \in \mathcal{N}} \mathbb{E} \left\{ \sum_{i \in I_n(t)} a_n^i(t) \tilde{Q}_n^F(t) | \Theta(t) \right\} \\ & + \sum_{n \in \mathcal{N}} \mathbb{E} \left\{ \sum_{i \in \mathbf{Q}_n^F(t)} s^i \alpha_n^i(t) [Q_n^B(t) - \tilde{Q}_n^F(t)] \right. \\ & \left. + V \alpha_n^i(t) [\rho_d d_n^{i,C} + \rho_e e_n^{i,C}] | \Theta(t) \right\} \\ & + \sum_{n \in \mathcal{N}} \sum_{m \in N_n} \mathbb{E} \left\{ \sum_{i \in \mathbf{Q}_n^F(t)} \beta_{nm}^i(t) \left[V \rho_d \left(\frac{s^i}{b_{nm}(t)} + T_{nm}(t) \right) \right. \right. \\ & \left. \left. + V \rho_e \frac{s^i}{b_{nm}(t)} P_t - s^i \tilde{Q}_n^F(t) - s^i [Q_n^F(t) - \theta_n]^1 \right] | \Theta(t) \right\}, \end{aligned} \quad (21)$$

where $[x]^1 = x$ if $x \leq 0$, and $[x]^1 = 0$ if $x > 0$. Any feasible offloading decision algorithm that produces the optimal solution to **P3** is within a constant gap from the minimum of the RHS of (16). See Appendix B for the derivation process of **P3**, available in the online supplemental material.

Note that the optimization object of **P3** is the sum of all nodes $n \in \mathcal{N}$ and the variable $a_n(t)$, $\alpha_n(t)$ and $\beta_n(t)$ can be decoupled from the optimization object. Therefore, **P3** can be addressed by dealing with the several subproblems separately. We further decompose **P3** into $|\mathcal{N}|$ independent subproblems, each subproblem aims at minimizing n 's own one-slot one-hop offloading overhead. Satellites coordinate their offloading strategies in an autonomous way over time to jointly minimize the time expectation of the total system overhead.

Based on the analysis above, we propose the distributed satellite peer computation offloading scheme. The task offloading scheme on a single satellite includes three steps: task admission, task scheduling, and task processing.

- **Task Admission:** Task admission decisions are made by minimizing the first term of (21). In each slot, every satellite independently solves

$$\min_{a_n(t)} \sum_{i \in I_n(t)} a_n^i(t) [Q_n^F(t) - \theta_n]. \quad (22)$$

The optimal solution reduces to a simple threshold rule

$$a_n^i(t) = \begin{cases} 1, & Q_n^F(t) \leq \theta_n, \\ 0, & \text{otherwise.} \end{cases} \quad (23)$$

where θ_n is an offline parameter related to network stability and resource utilization, explained in Section IV-D. The formula of (23) means that tasks are admitted only when the workload of the access satellite forwarding queue is under threshold.

- **Task Scheduling:** Satellite makes task scheduling decisions by independently minimizing the second and third terms in the subproblems of (21).

$$\begin{aligned} & \min_{s_n(t)} \sum_{i \in \mathbf{Q}_n^F(t)} \alpha_n^i(t) \\ & \times \left[s^i Q_n^B(t) - s^i \tilde{Q}_n^F(t) + V (\rho_d d_n^{i,C} + \rho_e e_n^{i,C}) \right] \\ & + \sum_{m \in N_n} \beta_{nm}^i(t) \left[V \rho_d \left(\frac{s^i}{b_{nm}(t)} + T_{nm}(t) \right) \right. \\ & \left. + V \rho_e \frac{s^i}{b_{nm}(t)} P_t - s^i \tilde{Q}_n^F(t) - s^{\max} [Q_n^F(t) - \theta_n]^1 \right] \\ & \text{s.t. (12a)–(12d).} \end{aligned} \quad (24)$$

Note that the decision is made based on the collection of the neighbors' future workload information. The scheduling process (24) is executed on every satellite at any slot and it is essentially an assignment problem: given a set of newly incoming tasks and current network status, how to assign

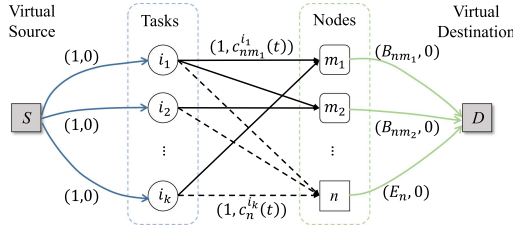


Fig. 3. An example of CMCMF graph construction.

resources for each task with the minimum system overhead under constraints.

- *Task processing*: The satellite processes all tasks in $\mathbf{Q}_n^B(t)$, which are accepted to $\mathbf{Q}_n^B$ at the end of $t - 1$. The allocation of processing rate is as (3).

Now, we describe the task scheduling step in detail. We model the task scheduling problem as a capacity-constraint minimum cost maximum flow problem (CMCMF). The input of CMCMF is a set of flows (task requests queued in $\mathbf{Q}_n^F(t)$ to be assigned in this paper's context) and network graph (the status of satellite and its neighbors). The output is the flow routing plans (task scheduling decisions) and the minimum cost (the optimization object of (24)). The graph of CMCMF is constructed as Fig. 3. First, let $V_{nt} = \{\mathbf{Q}_n^F(t) \cup N_n \cup \{n, S, D\}\}$ denote the set of vertices. S and D is a pair of virtual source and destination. There are two layers of vertices between S and D . Each task $i \in \mathbf{Q}_n^F(t)$ corresponds to a vertex in the first layer. The second layer consists of nodes in N_n and satellite n itself. Next, let ε_{nt} be the set of undirected edges. S establishes an edge of capacity 1 and cost 0 with each vertex in the first layer, which means each task can be offloaded to at most one satellite at one slot, coinciding with (12a). The rule of establishing edges between the first and second layer is as following. Each vertex i_k establishes an edge of capacity 1 with a vertex m_j in the second layer if constraint (12b) is satisfied. The cost of such an edge is same as the cost of peer offloading, which is $c_{nm}^i(t) = V\rho_d(\frac{s^i}{b_{nm}(t)} + T_{nm}(t)) + V\rho_e\frac{s^i}{b_{nm}(t)}P_t - s^i\tilde{Q}_n^F(t) - s^{\max}\lceil Q_n^F(t) - \theta_n \rceil^1$. Each vertex i_k must connect to n with capacity 1 and local processing cost $c_n^i(t) = s^iQ_n^B(t) - s^i\tilde{Q}_n^F(t) + V(\rho_d d_n^{i,C} + \rho_e e_n^{i,C})$. Every vertex m_j in the second layer connects to D with capacity B_{nm_j} and cost 0 under constraint (12c). Similarly, vertex n is connected to D with capacity E_n and cost 0 due to limited processing capacity (12d). Generally, by reducing (24) to a CMCMF problem, a lot of algorithms can be used to solve this problem in polynomial time, such as the Successive Shortest Path (SSP) algorithm and so on.

The complete peer offloading scheme on an individual satellite is shown in Algorithm 2.

E. Properties Analysis

Complexity: The distributed online satellite peer offloading algorithm includes three steps. The running time of the first step of task admission (line 2–4) is $O(I_n(t))$. In the second step (line 5–10), the algorithm considers $|N_n| + 1$ candidate scheduling strategies for each task queued in $\mathbf{Q}_n^F(t)$. The complexity of predicting $d_m^{i,C}$ and $e_m^{i,C}$ by using Algorithm 1 (line 9–15) is $O(|T_t|)$. The algorithm applies

Algorithm 2: Distributed Satellite Peer Offloading Scheme on Node n at Slot t .

- 1: Update $\mathbf{Q}_n^F(t)$ and $\mathbf{Q}_n^B(t)$ by (1) and (2);
- 2: **for** each task $i \in I_n(t)$ **do**
- 3: Get the task admission decision by (23);
- 4: **end for**
- 5: **for** each task $i \in \mathbf{Q}_n^F(t)$ **do**
- 6: **for** each node $m \in N_n \cup \{n\}$ **do**
- 7: Apply Algorithm 1 to obtain $d_m^{i,C}$ and $e_m^{i,C}$;
- 8: **end for**
- 9: **end for**
- 10: Apply SSP algorithm to derive task scheduling decisions $\alpha_n(t)$ and $\beta_{nm}(t)$;
- 11: **for** each task $i \in \mathbf{Q}_n^B(t)$ **do**
- 12: Process the task according to (3) and (4);
- 13: **end for**

SSP to obtain the final peer offloading decisions with the running time of $\min\{O(v^2 f^*), O(v^3 c^*)\}$, where $v = |N_n| + 3 + |\mathbf{Q}_n^F(t)|$ is the total number of vertices based on V_{nt} , f^* is the derived maximum flow, and c^* is the corresponding minimum cost. Hence, the total complexity of task scheduling is $O(|\mathbf{Q}_n^F(t)|(|N_n| + 1)|T_t| + \min\{O(v^2 f^*), O(v^3 c^*)\})$. In the task processing step, the amount of workloads is computed with complexity of $O(|\mathbf{Q}_n^B(t)|)$. Therefore, the overall complexity of distributed online satellite peer offloading algorithm is $\min\{O(I_n(t) + |\mathbf{Q}_n^F(t)|(|N_n| + 1)|T_t| + v^2 f^* + |\mathbf{Q}_n^B(t)|), O(I_n(t) + |\mathbf{Q}_n^F(t)|(|N_n| + 1)|T_t| + v^3 c^* + |\mathbf{Q}_n^B(t)|)\}$. The algorithm can find a near-optimal solution in polynomial time.

Close-to-Optimality: In this part, we rigorously prove that minimizing **P2** under the framework of Lyapunov optimization can achieve close-to-optimal system overhead compared to **P1** while guaranteeing system stability in the case of dynamic task arrivals.

First, we define the perturbation parameter as

$$\theta_n \triangleq 2(E_n + [N - 1] B^{\max}). \quad (25)$$

which is an offline parameter that depends on each satellite's computation and communication capacity. Such feature is favorable to practical implementations.

Lemma 2: Suppose that $\forall n \in \mathcal{N}, Q_n^F(0) = \theta_n$, and $Q_n^B(0) = 0$. Assume we can obtain admission decisions $\pi(t)$ and offloading decisions $s(t)$ by Algorithm 2. In that case, we denote the system overhead obtained by Algorithm 2 as $\hat{C}(t)$ for each time slot, and the backlogs of forwarding queue and computing queue are $\hat{Q}_n^F(t)$ and $\hat{Q}_n^B(t)$, respectively. Following the distributed online satellite peer offloading decision-making scheme, the long-term system stability satisfies

$$\begin{aligned} & \limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{n=1}^N \mathbb{E} \left\{ \hat{Q}_n^B(t) + \hat{Q}_n^F(t) \right\} \\ & \leq \sum_{n=1}^N \theta_n + \frac{B_1 + V(C^{\max} - C^*)}{\varepsilon}, \end{aligned} \quad (26)$$

and the long-term system overhead satisfies

$$\limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} \{ \hat{C}(t) \} \leq C^* + \frac{B_1}{V}, \quad (27)$$

where C^* is the optimal solution of **P1**, $C^{\max} = NE^{\max}(\rho_d d^{C, \max} + \rho_e e^{C, \max}) + 4N[\rho_d(s^{\max}/b^{\min} + T^{\max}) + \rho_e s^{\max} P^{\max}/b^{\min}]$, and $\varepsilon > 0$ is a constant which represents the long-term resource surplus achieved by some stationary strategy. Here we assume that a satellite establishes finite number of links with neighbors. The proof of Lemma 2 is given in Appendix C, available in the online supplemental material.

The above lemma demonstrates an $[O(1/V), O(V)]$ overhead-stability tradeoff. Algorithm 2 asymptotically achieves the optimal performance of the offline problem **P1** by letting $V \rightarrow \infty$. However, the optimal system overhead is achieved at the price of a larger system congestion, as a larger system congestion deteriorates the task processing delay and postpones the convergence. The long-term system stability bound in (26) implies that the time-average queue length grows linearly with V .

Robustness Against Workload Prediction Errors: Although we have already analyzed the prediction error of Algorithm 1, we have to figure out the effect on the system overhead and stability when the predicted workload $Q_n^{B^*}(t)$ differs from the actual.

Lemma 3: Suppose there exists a constant ξ such that $|Q_n^{B^*}(t) - Q_n^B(t)| \leq \xi$ holds for all slots $t > 0$. Under the prediction error ξ , the long-term system stability satisfies

$$\begin{aligned} & \limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{n=1}^N \mathbb{E} \{ Q_n^B(t) + Q_n^F(t) \} \\ & \leq \sum_{n=1}^N \theta_n + \frac{B_3 + V(C^{\max} - C^*)}{\varepsilon}, \end{aligned} \quad (28)$$

and the long-term system overhead satisfies

$$\limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} \{ \hat{C}(t) \} \leq C^* + \frac{B_3}{V}, \quad (29)$$

where $B_3 = B_1 + \xi \sum_{n \in \mathcal{N}} [s^{\max} E_n + \frac{Q_n^{B, \max} r_n^{\max}}{s_{\min} h_{\min}}]$.

See [25] for the details of Lemma 3. Comparing Lemma 3 with Lemma 2, we conclude that the upper bounds of long-term system overhead and queue length are larger under inaccurate workload predictions. However, the robustness of Algorithm 2 is still guaranteed. Besides, with the prediction error, a larger V is desired to achieve the same average system overhead as that with exact workload information, but at the expense of decreased stability.

V. SIMULATION RESULTS AND DISCUSSIONS

In this section, we discuss the main findings of this paper. We evaluate the proposed multi-hop satellite peer computation offloading scheme under various system settings and give insights on the performance compared with baseline schemes.

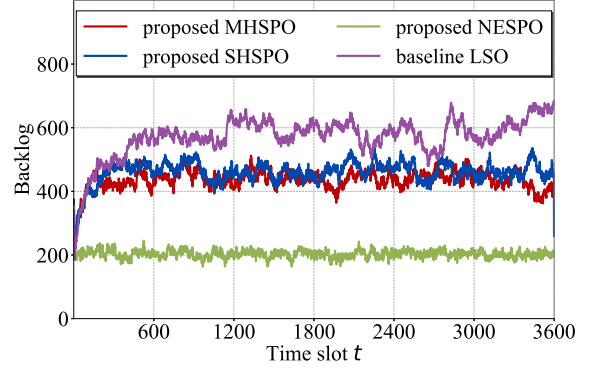


Fig. 4. Comparison of system backlogs over time.

A. Simulation Settings

We assume that the MEC network is deployed in a 1,070 km low-orbit satellite constellation with an inclination of 87° . The scale of the constellation should be large, such that multiple collaborations among MEC-enhanced satellites can be exploited. We set 192 satellites on 16 orbits, with 12 satellites on each orbit plane. The simulation's operational time is 3,600 seconds, each slot lasts one second for making satellite peer offloading decisions. The computation capability of satellites is 2 Gcycles/s [14]. The local computing capacity of each satellite E_n is 6 tasks/slot [25]. The transmission rate of inter-satellite links are uniformly distributed in [100 M, 300M] bit/s. The inter-satellite transmission power is 0.1 W and we set $\kappa = 10^{-26}$ [14]. The propagation delay among neighbor satellites are calculated as the straight-line distance divided by the speed of light.

We generate computation demands based on real Internet usage [32], [53]. Specifically, the earth is divided into $15^\circ \times 15^\circ$ geographical zones, so in total there are 288 such zones. The task arrival rate of each zone is proportional to the Internet users distribution data in [54]. Additionally, to simulate computation demands of devices in remote areas, the task generation of each zone follows a Poisson process with arrival rate $\pi_n(t) \in [0, 2]$ tasks/sec. The total computation task generation is the sum of the above two parts. For each task, its size is uniformly distributed in [10 M, 50M] bits. We assume the required number of CPU cycles for processing one bit of each task is randomly drawn from [100, 300]. The deadline of task is randomly generated within a range of [1, 12] seconds.

The performances of *multi-hop satellite peer offloading (MH-SPO)* is compared with three benchmarks:

1) *Single-hop satellite peer offloading (SH-SPO)*: Tasks are offloaded to at most one-hop peer satellites. The access satellite chooses the best one-hop offloading decision in the same approach as MH-SPO: if offloading to one-hop neighbors is not beneficial, the access satellite will process the task locally.

2) *No-energy satellite peer offloading (NESPO)*: The decision-making process of NESPO is mostly the same as that of MH-SPO, the only difference is that NESPO does not take the system energy overhead into account, which means $\rho_e = 0$.

3) *Local satellite offloading (LSO)*: No satellite peer offloading is enabled in LSO. For each task, the access satellite will process it locally.

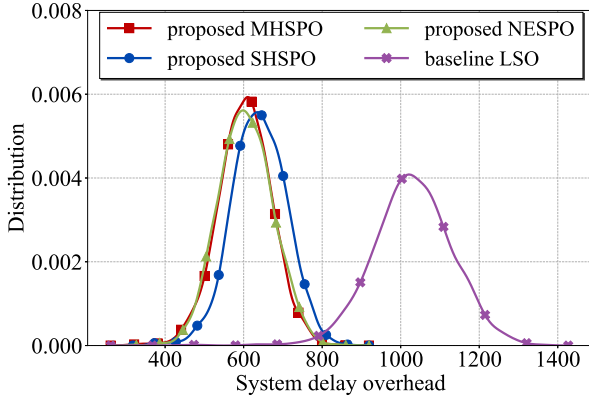


Fig. 5. Comparison of the distribution of system delay overhead over time.

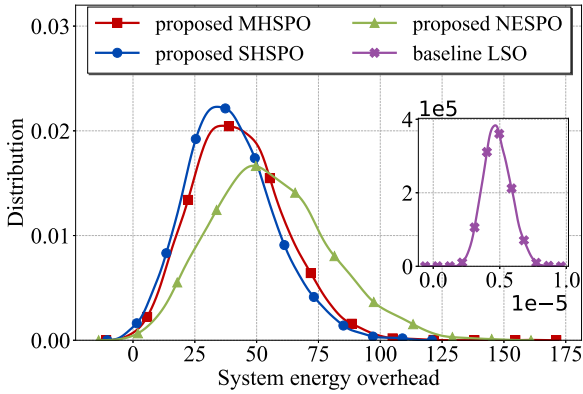


Fig. 6. Comparison of the distribution of system energy overhead over time.

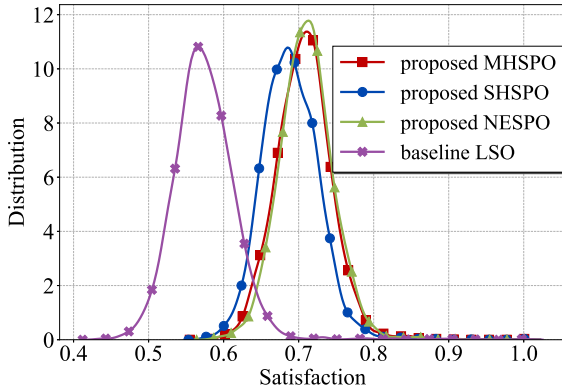


Fig. 7. Comparison of the distribution of users' satisfaction over time.

B. Run-Time Performance Evaluation

Figs. 4–7 show the long-term system performance obtained by running four schemes with $V = 100$ and $\rho_d = 0.005$, $\rho_e = 0.995$. We mainly focus on four metrics: the system backlogs over time in Fig. 4, the system delay overhead in Fig. 5, the system energy overhead in Fig. 6, and the users' satisfaction in Fig. 7. Note that Figs. 5 and 6 show the histogram distributions of the system metric data over time slots, and Fig. 7 shows the probability density distributions of the number of tasks

completed before their deadlines divided by the total number of tasks finished at each slot.

It can be observed that without peer offloading the MEC-enabled satellite system bears a high backlog, a large system delay overhead, and a low users' satisfaction, since single satellites can be easily overloaded due to spatially and temporally heterogeneous task arrival pattern. The system energy overhead of LSO is the lowest among the four schemes because the task transmission is not permitted in LSO, which accounts for the majority of system energy consumption. By contrast, other three schemes with peer offloading enabled (MHSPo, SHSPo, NESPO) achieve much lower backlogs, much smaller system delay overhead, and better users' satisfaction. Specifically, NESPO achieves the lowest system backlog and the best delay performance since it is designed to minimize the delay overhead, so it can thoroughly balance the workloads regardless of the energy consumption. Therefore, NESPO incurs a larger energy consumption as shown in Fig. 6. Surprisingly, the users' satisfaction of NESPO is not much higher than that of MHSPo. The reason is that, in order to balance the load, NESPO offloads tasks to satellites more hops away. The time that tasks wait in intermediate nodes for next decisions at each slot is not included in the calculation of system delay overhead, but is considered in task completion delay.

The main purpose of MHSPo is to constrain the system backlogs while minimizing the system overhead. As can be observed in Figs. 4 and 5, MHSPo achieves the lowest system backlog and the smallest system delay overhead among MHSPo, SHSPo, and LSO, which means that the multi-hop offloading way balance the load better. Although the energy consumption of SHSPo is lower than MHSPo in Fig. 6, the users' satisfaction of MHSPo is higher in Fig. 7. Hence, we conclude that the proposed MHSPo scheme can well balance the system backlog and the system overhead, while meeting the requirements of most users.

C. System Dynamics

Figs. 8 and 9 show the system backlog and the system overhead from the 500th to 550th time slot, respectively. Considering the fact that a newly admitted arrival cannot arrive at Q^B until at least one slot later, we show the backlog of Q^B from the 501th to 551th time slot in Fig. 8(a) to clearly demonstrate how Q^B varies with the arrival.

We see that the system backlog of Q^F is mainly decided by the total task arrival rate in the network which varies across the time slots. Usually, a larger task arrival rate will result in a higher Q^F backlog. But the value fluctuation of Q^F is smaller than that of the system arrival due to the task admission excuted on each satellite. Moreover, the trend of Q^B is roughly the same as that of the system arrival. But a sudden increase or decrease in arrival does not always brings surge or fall in Q^B as presented at slot 509 and 534 in Fig. 8(a). The reason is that the value of Q^B backlog also depends on the departure of tasks. Only a low (high) level of arrivals persisting for a period of slots can bring about a downward (upward) trend of Q^B (as shown from slot 511 to 516, and from slot 530 to 533). In addition, the overhead

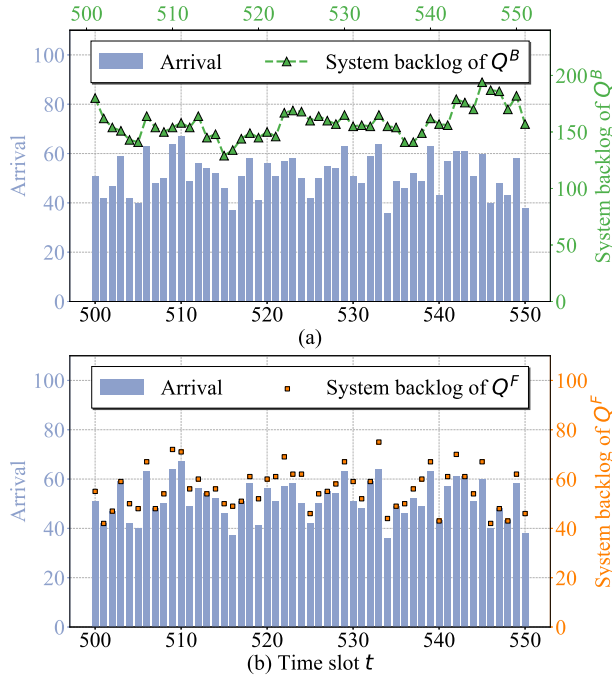


Fig. 8. System dynamics (system backlogs).

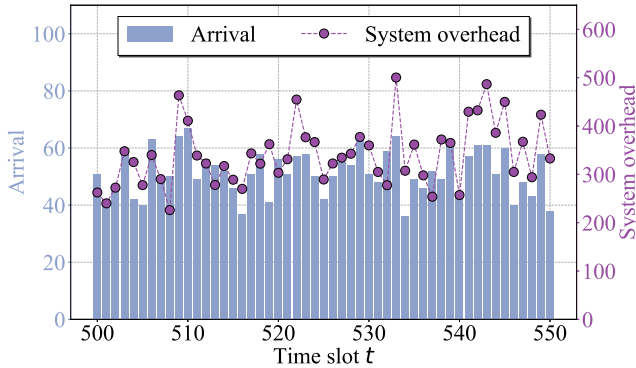
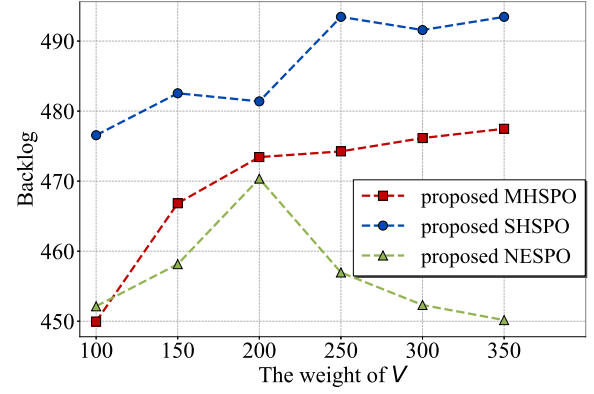
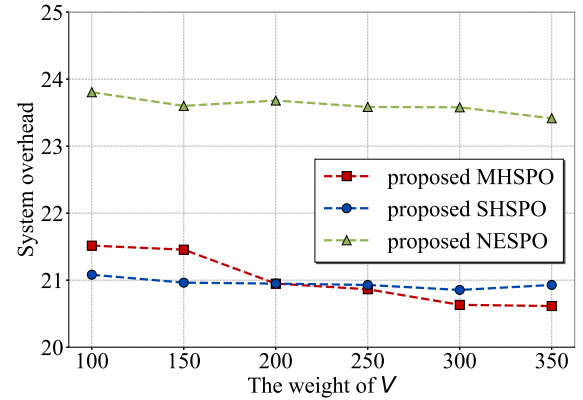


Fig. 9. System dynamics (system overhead).

of the system fluctuates with the system arrival, a higher system arrival always brings a larger system overhead as depicted in Fig. 9.

D. Impact of Parameter V

Figs. 10 and 11 show the impact of parameter V on the performance of MHSPo, SHSPo, and NESPO. From Fig. 10, we can see that the average system backlogs of MHSPo and SHSPo increase with V , since more attention is paid to system overhead. MHSPo always has less backlogs than SHSPo. The backlogs of NESPO does not consistently rises with V because the goal of minimizing the system delay overhead coincides with minimizing the backlogs. As shown in Fig. 11, the system overhead under the three mechanisms decreases with V and MHSPo achieves the lowest overhead. The reason lies in the more emphasis highlighted on the system overhead. With a larger V , the SHSPo cannot achieve the same amount of system overhead decline as MHSPo because it is only allowed to schedule load balancing within one-hop range.


 Fig. 10. System backlogs with varying V .

 Fig. 11. System overhead with varying V .

These two figures present a $[O(1/V), O(V)]$ trade-off between the long-term system overhead and the long-term system backlogs, which is consistent with our theoretical analysis. With a larger V , MHSPo emphasizes more on the system overhead and is less concerned with the system backlogs. As V grows to the infinity, MHSPo is able to achieve the optimal delay overhead, but it is hard to define an optimal value for V since a lower system overhead is achieved at the cost of larger backlogs. However, it still offers a guideline for picking an appropriate value of V . In this particular simulation, the network operator is recommended to choose, for example, $V = 100$ for two reasons: (i) MHSPo has already achieved close-to-optimal delay and little improvement is available by increasing V ; (ii) the system backlog is much smaller when $V = 100$ compared to the system backlogs with higher V .

VI. CONCLUSION

In this paper, we proposed a multi-hop computation peer offloading scheme in MEC-enabled satellite networks where unbalanced and heterogeneous task arrival pattern is considered. We formulated a novel MHSPo problem to jointly optimize the long-term delay and energy consumption while confining the system backlogs. A Lyapunov-based framework was introduced to optimize the system performance online. The proposed scheme allows autonomous decision-making and provides a provable performance guarantee. We also showed that the proposed scheme balances the system overheads and backlogs well.

In the future, we will consider the online learning technique with less prediction regret to further improve the system performance. Moreover, compared with MEC-enhanced satellites, terrestrial data centers have more powerful computing capabilities but suffer long RTT latency. A satellite-terrestrial combined two-layer computation offloading scheme is a promising research direction.

REFERENCES

- [1] J. Liu, Y. Shi, Z. M. Fadlullah, and N. Kato, "Space-air-ground integrated network: A survey," *IEEE Commun. Surveys Tut.*, vol. 20, no. 4, pp. 2714–2741, Fourth Quarter 2018.
- [2] O. Kodheli et al., "Satellite communications in the new space era: A survey and future challenges," *IEEE Commun. Surveys Tut.*, vol. 23, no. 1, pp. 70–109, First Quarter 2021.
- [3] H. Guo, J. Li, J. Liu, N. Tian, and N. Kato, "A survey on space-air-ground-sea integrated network security in 6G," *IEEE Commun. Surveys Tut.*, vol. 24, no. 1, pp. 53–87, First Quarter 2022.
- [4] S. Duan et al., "Distributed artificial intelligence empowered by end-edge-cloud computing: A survey," *IEEE Commun. Surveys Tut.*, vol. 25, no. 1, pp. 591–624, First Quarter 2023.
- [5] Z. Zhang, W. Zhang, and F.-H. Tseng, "Satellite mobile edge computing: Improving QoS of high-speed satellite-terrestrial networks using edge computing techniques," *IEEE Netw.*, vol. 33, no. 1, pp. 70–76, Jan./Feb. 2019.
- [6] B. Shang, Y. Yi, and L. Liu, "Computing over space-air-ground integrated networks: Challenges and opportunities," *IEEE Netw.*, vol. 35, no. 4, pp. 302–309, Jul./Aug. 2021.
- [7] K. Wei, Q. Tang, J. Guo, M. Zeng, Z. Fei, and Q. Cui, "Resource scheduling and offloading strategy based on LEO satellite edge computing," in *Proc. IEEE 94th Veh. Technol. Conf.*, 2021, pp. 1–6.
- [8] S. Cao, H. Han, J. Wei, Y. Zhao, S. Yang, and L. Yan, "Space cloud-fog computing: Architecture, application and challenge," in *Proc. 3rd ACM Int. Conf. Comput. Sci. Appl. Eng.*, 2019, Art. no. 63.
- [9] R. Xie, Q. Tang, Q. Wang, X. Liu, F. R. Yu, and T. Huang, "Satellite-terrestrial integrated edge computing networks: Architecture, challenges, and open issues," *IEEE Netw.*, vol. 34, no. 3, pp. 224–231, May/Jun. 2020.
- [10] B. Denby and B. Lucia, "Orbital edge computing: Nanosatellite constellations as a new class of computer system," in *Proc. 25th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2020, pp. 939–954.
- [11] D. Bhattacharjee, S. Kassing, M. Licciardello, and A. Singla, "In-orbit computing: An outlandish thought experiment?," in *Proc. 19th ACM Workshop Hot Topics Netw.*, 2020, pp. 197–204.
- [12] T. Chen et al., "Learning-based computation offloading for IoRT through ka/Q-band satellite-terrestrial integrated networks," *IEEE Internet Things J.*, vol. 9, no. 14, pp. 12056–12070, Jul. 2022.
- [13] G. Cui, X. Li, L. Xu, and W. Wang, "Latency and energy optimization for MEC enhanced SAT-IoT networks," *IEEE Access*, vol. 8, pp. 55915–55926, 2020.
- [14] Z. Song, Y. Hao, Y. Liu, and X. Sun, "Energy-efficient multiaccess edge computing for terrestrial-satellite Internet of Things," *IEEE Internet Things J.*, vol. 8, no. 18, pp. 14202–14218, Sep. 2021.
- [15] Q. Tang, Z. Fei, B. Li, and Z. Han, "Computation offloading in LEO satellite networks with hybrid cloud and edge computing," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 9164–9176, Jun. 2021.
- [16] S. Zhang, G. Cui, Y. Long, and W. Wang, "Joint computing and communication resource allocation for satellite communication networks with edge computing," *China Commun.*, vol. 18, no. 7, pp. 236–252, 2021.
- [17] C. Li, Y. Zhang, X. Hao, and T. Huang, "Jointly optimized request dispatching and service placement for MEC in LEO network," *China Commun.*, vol. 17, no. 8, pp. 199–208, 2020.
- [18] Y. Wang, J. Yang, X. Guo, and Z. Qu, "A game-theoretic approach to computation offloading in satellite edge computing," *IEEE Access*, vol. 8, pp. 12510–12520, 2020.
- [19] M. A. Islam, S. Ren, G. Quan, M. Z. Shakir, and A. V. Vasilakos, "Water-constrained geographic load balancing in data centers," *IEEE Trans. Cloud Comput.*, vol. 5, no. 2, pp. 208–220, Second Quarter 2017.
- [20] Z. Liu, M. Lin, A. Wierman, S. Low, and L. L. H. Andrew, "Greening geographical load balancing," *IEEE/ACM Trans. Netw.*, vol. 23, no. 2, pp. 657–671, Apr. 2015.
- [21] L. Chen, S. Zhou, and J. Xu, "Computation peer offloading for energy-constrained mobile edge computing in small-cell networks," *IEEE/ACM Trans. Netw.*, vol. 26, no. 4, pp. 1619–1632, Aug. 2018.
- [22] J. Ren et al., "An efficient two-layer task offloading scheme for MEC system with multiple services providers," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1519–1528.
- [23] Z. Jing, Q. Yang, Y. Wu, M. Qin, K. Sup Kwak, and X. Wang, "Adaptive cooperative task offloading for energy-efficient small cell MEC networks," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2022, pp. 292–297.
- [24] K. Wang, Y. Zhou, J. Li, L. Shi, W. Chen, and L. Hanzo, "Energy-efficient task offloading in massive MIMO-aided multi-pair fog-computing networks," *IEEE Trans. Commun.*, vol. 69, no. 4, pp. 2123–2137, Apr. 2021.
- [25] Y. Li, X. Wang, X. Gan, H. Jin, L. Fu, and X. Wang, "Learning-aided computation offloading for trusted collaborative mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 19, no. 12, pp. 2833–2849, Dec. 2020.
- [26] L. Hu, Y. Tian, J. Yang, T. Taleb, L. Xiang, and Y. Hao, "Ready player one: UAV-clustering-based multi-task offloading for vehicular VR/AR gaming," *IEEE Netw.*, vol. 33, no. 3, pp. 42–48, May/Jun. 2019.
- [27] Z. Xiao et al., "Vehicular task offloading via heat-aware MEC cooperation using game-theoretic method," *IEEE Internet Things J.*, vol. 7, no. 3, pp. 2038–2052, Mar. 2020.
- [28] J. Wang, K. Zhu, B. Chen, and Z. Han, "Distributed clustering-based cooperative vehicular edge computing for real-time offloading requests," *IEEE Trans. Veh. Technol.*, vol. 71, no. 1, pp. 653–669, Jan. 2022.
- [29] P. Dai, K. Hu, X. Wu, H. Xing, F. Teng, and Z. Yu, "A probabilistic approach for cooperative computation offloading in MEC-assisted vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 899–911, Feb. 2022.
- [30] Y. Wu, K. Ni, C. Zhang, L. P. Qian, and D. H. K. Tsang, "NOMA-assisted multi-access mobile edge computing: A joint optimization of computation offloading and time allocation," *IEEE Trans. Veh. Technol.*, vol. 67, no. 12, pp. 12244–12258, Dec. 2018.
- [31] S. Duan et al., "MOTO: Mobility-aware online task offloading with adaptive load balancing in small-cell MEC," *IEEE Trans. Mobile Comput.*, early access, Nov. 08, 2022, doi: [10.1109/TMC.2022.3202720](https://doi.org/10.1109/TMC.2022.3202720).
- [32] Y. Yang, M. Xu, D. Wang, and Y. Wang, "Towards energy-efficient routing in satellite networks," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 12, pp. 3869–3886, Dec. 2016.
- [33] N. Cheng et al., "Space/aerial-assisted computing offloading for IoT applications: A learning-based approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1117–1129, May 2019.
- [34] M. De Sanctis, E. Cianca, G. Araniti, I. Bisio, and R. Prasad, "Satellite communications supporting internet of remote things," *IEEE Internet Things J.*, vol. 3, no. 1, pp. 113–123, Feb. 2016.
- [35] W.-C. Chien, C.-F. Lai, M. S. Hossain, and G. Muhammad, "Heterogeneous space and terrestrial integrated networks for IoT: Architecture and challenges," *IEEE Netw.*, vol. 33, no. 1, pp. 15–21, Jan./Feb. 2019.
- [36] R. Radhakrishnan, W. W. Edmonson, F. Afghah, R. M. Rodriguez-Orsorio, F. Pinto, and S. C. Burleigh, "Survey of inter-satellite communication for small satellite systems: Physical layer to network layer view," *IEEE Commun. Surveys Tut.*, vol. 18, no. 4, pp. 2442–2473, Fourth Quarter 2016.
- [37] B. d'Humières et al., "The C3PO project: A laser communication system concept for small satellites," in *Proc. Free-Space Laser Commun. Atmospheric Propag. XXIX*, SPIE, 2017, pp. 285–297.
- [38] H. Tomio et al., "Assembly and integration of optical downlink terminal vsota on microsatellite risesat," in *Proc. Int. Conf. Space Opt.*, SPIE, 2019, pp. 2147–2157.
- [39] X. Xiaobin, Z. Hui, L. Chang, F. Cunku, L. Zhongjun, and W. Shangguang, "On the aggregated resource management for satellite edge computing," in *Proc. IEEE Int. Conf. Commun.*, 2021, pp. 1–6.
- [40] H. Fang, Y. Jia, Y. Wang, Y. Zhao, Y. Gao, and X. Yang, "Matching game based task offloading and resource allocation algorithm for satellite edge computing networks," in *Proc. Int. Symp. Netw. Comput. Commun.*, 2022, pp. 1–5.
- [41] Y. Wang, J. Zhang, X. Zhang, P. Wang, and L. Liu, "A computation offloading strategy in satellite terrestrial networks with double edge computing," in *Proc. IEEE Int. Conf. Commun. Syst.*, 2018, pp. 450–455.
- [42] D. Calzolaio, F. Curreli, J. Duncan, A. Moorhouse, G. Perez, and S. Voegt, "EDRS-C—The second node of the European data relay system is in orbit," *Acta Astronautica*, vol. 177, pp. 537–544, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0094576520304707>
- [43] D. M. Cornwell, "NASA's optical communications program for 2017 and beyond," in *Proc. IEEE Int. Conf. Space Opt. Syst. Appl.*, 2017, pp. 10–14.

- [44] A. U. Chaudhry and H. Yanikomeroglu, "Temporary laser inter-satellite links in free-space optical satellite networks," *IEEE Open J. Commun. Soc.*, vol. 3, pp. 1413–1427, Aug. 2022.
- [45] H. Kaushal and G. Kaddoum, "Optical communication in space: Challenges and mitigation techniques," *IEEE Commun. Surveys Tut.*, vol. 19, no. 1, pp. 57–96, First Quarter 2017.
- [46] Q. Yang, L. Tan, and J. Ma, "Doppler characterization of laser inter-satellite links for optical LEO satellite constellations," *Opt. Commun.*, vol. 282, no. 17, pp. 3547–3552, 2009.
- [47] D. Zhou, M. Sheng, R. Liu, Y. Wang, and J. Li, "Channel-aware mission scheduling in broadband data relay satellite networks," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 5, pp. 1052–1064, May 2018.
- [48] Neely and J. Michael, "Stochastic network optimization with application to communication and queueing systems," *Synth. Lect. Commun.*, vol. 3, no. 1, pp. 1–211, 2010.
- [49] X. Zhang, C. Wu, Z. Li, and F. C. Lau, "Proactive VNF provisioning with multi-timescale cloud resources: Fusing online learning and online optimization," in *Proc. IEEE Conf. Comput. Commun.*, 2017, pp. 1–9.
- [50] P. Joulani, A. Gyorgy, and C. Szepesvari, "Online learning under delayed feedback," in *Proc. Int. Conf. Mach. Learn.*, 2013, pp. 1453–1461.
- [51] K. Quanrud and D. Khashabi, "Online learning with adversarial delays," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 1270–1278.
- [52] N. Chen, A. Agarwal, A. Wierman, S. Barman, and L. L. Andrew, "Online convex optimization using predictions," in *Proc. ACM SIGMETRICS Int. Conf. Meas. Model. Comput. Syst.*, 2015, pp. 191–204.
- [53] A. Papa, T. De Cola, P. Vizarreta, M. He, C. Mas Machuca, and W. Kellerer, "Dynamic SDN controller placement in a LEO constellation satellite network," in *Proc. IEEE Glob. Commun. Conf.*, 2018, pp. 206–212.
- [54] Internet usage statistics, 2022. [Online]. Available: <https://www.internetworldstats.com/stats.htm>



Xinyuan Zhang received the BS degree in communication engineering from the Beijing University of Posts and Telecommunications (BUPT), China, in 2019. She is currently working toward the PhD degree with the State Key Laboratory of Networking and Switching Technology, in BUPT. She is a visiting PhD student with the Information Systems Technology and Design Pillar, Singapore University of Technology and Design, Singapore, from 2022. Her current research interests include satellite-terrestrial integrated networks, edge networks, and deterministic networks.



Jiang Liu received the BS degree in electronics engineering from the Beijing Institute of Technology, Beijing, China, in 2005, the MS degree in communication and information system from Zhengzhou University, Zhengzhou, China, in 2009, and the PhD degree from the Beijing University of Posts and Telecommunications, Beijing, in 2012. He is currently a professor with the Beijing University of Posts and Telecommunications. His current research interests include network architecture, network virtualization, satellite networking, software-defined networking (SDN), information-centric networking (ICN), and network testbed.



Ran Zhang (Member, IEEE) received the PhD degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 2021. He is currently a post-doctoral research fellow with the State Key Laboratory of Networking and Switching Technology, BUPT. His research interests include caching, computing, communication integration, satellite-terrestrial integrated networks, and artificial intelligence.



Yudong Huang received the BS degree in communication engineering from the Beijing University of Posts and Telecommunications (BUPT), China, in 2019. He is currently working toward the PhD degree with the State Key Laboratory of Networking and Switching Technology, in BUPT. He was a visiting PhD student with the School of Computer Science and Engineering, Nanyang Technological University, Singapore, from 2022 to 2023. His current research interests include time-sensitive networks, deterministic networks, and network architecture.



Jincheng Tong received the MS degree from the China Academy of Space Technology (CAST), in 2006. He is currently working in China Academy of Space Technology (CAST), Beijing, China. His research interests include telecommunication satellite design, satellite payload design and space network technology.



Ning Xin received the PhD degree from the China Academy of Space Technology (CAST), in 2014. He is a researcher with the Institute of Telecommunication and Navigation Satellite, China Academy of Space Technology, Beijing, China. His research interests are spacecraft design, satellite payload design and space network system design.



Liang Liu received the PhD degree from the Beijing University of Aeronautics and Astronautics (BUAA), in 2017. He is a researcher with the Institute of Telecommunication and Navigation Satellite, China Academy of Space Technology, Beijing, China. His research interests are space network system design, satellite communication system design.



Zehui Xiong (Member, IEEE) received the PhD degree from Nanyang Technological University (NTU), Singapore. He is currently an Assistant Professor with the Singapore University of Technology and Design, and also an Honorary adjunct senior research scientist with Alibaba-NTU Singapore Joint Research Institute, Singapore. He was the visiting scholar with Princeton University and University of Waterloo. His research interests include wireless communications, Internet of Things, blockchain, edge intelligence, and Metaverse. He has published more than 150 research

papers in leading journals and flagship conferences and many of them are ESI Highly Cited Papers. He has won more than 10 Best Paper Awards in international conferences and is listed in the World's Top 2% Scientists identified by Stanford University. He is now serving as the editor or guest editor for many leading journals including *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Vehicular Technology*, *IEEE Internet of Things Journal*, *IEEE Transactions on Cognitive Communications and Networking*, *IEEE Transactions on Network Science and Engineering*, *International Surgery Journal*, *Journal of the Atmospheric Sciences*. He is the recipient of IEEE Early Career Researcher Award for Excellence in Scalable Computing, IEEE Technical Committee on Blockchain and Distributed Ledger Technologies Early Career Award, IEEE Internet Technical Committee Early Achievement Award, IEEE Best Land Transportation Paper Award, IEEE CSIM Technical Committee Best Journal Paper Award, IEEE SPCC Technical Committee Best Paper Award, IEEE VTS Singapore Best Paper Award, Chinese Government Award for Outstanding Students Abroad, and NTU SCSE Best PhD Thesis Runner-Up Award. He is now serving as the associate director of Future Communications R&D Programme.